DataFlow Sentinel

Master Project Specification

מפרט מאסטר לפרויקט End-to-End לחיזוי סיכון לכשל של Data Jobs לפני הרצה

An end-to-end platform that scores job failure risk before execution and turns predictions into operational decisions.

גרסה 3 - מבוססת על תוכנית העבודה הקיימת ועל הנחיות ה-Hybrid Working Mode

הוכן עבור: רוני מדמון | B.Sc. Information Systems - AI Specialization | August 2026

סטטוס: **SPECIFICATION FOR APPROVAL** - אין להתחיל **Implementation** לפני אישור ה-**Scope**.

00 | איך להשתמש במסמך

Scope contract + technical decision record לפרויקט כולו

המסמך הזה מחליף את תוכנית העבודה הקצרה כנקודת הייחוס המרכזית. הוא שומר את הכיוון הטוב שכבר הוגדר - Job-level prediction, מניעת Data Leakage, Temporal Split, business-value simulation ו-product layer - אך מחדד את יחידת החיזוי, ארכיטקטורת הנתונים, כללי ה-validation, גבולות ה-MVP ותנאי הכניסה ל-RAG.

מקורות שעליהם המסמך מבוסס

- DataFlow Sentinel - תוכנית עבודה מעודכנת, גרסה 2, מיקוד בערך עסקי (13 עמודים).
- Hybrid Working Mode + Master Specification prompt שסופק לפרויקט.

כלל אמינות

כל מספר, עמודה או הנחה עסקית שלא אושרו מהדאטה בפועל מסומנים כ- UNKNOWN - needs validation. המסמך לא ממציא Schema, עלויות או ביצועי מודל.

תוכן המסמך

Sections 01-14	Sections 15-27
01. Critical Review + Change Log	02. Executive Summary
03. Business Problem	04. Project Objectives
05. Success Criteria	06. Data Understanding
07. Prediction Design	08. Data Engineering Architecture
09. Data Layers	10. Data Quality Strategy
11. EDA Plan	12. Feature Engineering
13. Modeling Strategy	14. Evaluation Framework
15. Explainability	16. Business Value Simulator
17. Product Layer / API	18. Dashboard
19. RAG Architecture Evaluation	20. Repository Structure
21. Testing Strategy	22. Reproducibility
23. Logging & Monitoring	24. Project Roadmap
25. Portfolio / Interview Value	26. Final MVP Definition
27. Decision Log + Project State	

01 | Critical Review של התוכנית הקיימת

מה נשמר, מה דורש תיקון, ומה עדיין לא ידוע

התוכנית הקיימת חזקה בכיוון העסקי ובמניעת Leakage, אבל יש בה כמה נקודות שחשוב לתקן לפני קוד. אלו שינויים מהותיים בלבד - לא redesign לשם redesign.

נושא	החלטה נוכחית	החלטה מוצעת	סיבה	השפעה
Grain של Gold table	`fact_job_runs` מתואר כשורה אחת לכל `job_name`.	שורה אחת לכל Job run, עם `job_run_id` או מפתח מורכב שיאומת מה-Schema.	Job name יכול להופיע במספר הרצות; ערבוב הרצות יפגע ב-Target ובהיסטוריה.	משנה את מודל הנתונים, ה-split וה-historical features.
Failure rate	מופיעים 0.58% וגם 0.59% ברמת Task.	לא נשתמש בשיעור הזה כמדד project-level; נחשב מחדש לאחר Job aggregation.	יש אי-עקביות פנימית, ובכל מקרה ה-MVP מנבא Job failure.	מונע מסקנות מוטעות על imbalance.
Lead Time	מוצג כמדד עסקי מרכזי.	להוציא מ-MVP pre-run ולהחזיר רק אם קיימת נקודת prediction/alert אמיתית בזמן.	Lead Time דורש timestamps שמייצגים alert time מול failure time; זה לא מובטח ב-`batch_task`.	מקטין scope ומונע metric לא אמין.
`batch_instance`	מופיע בארכיטקטורה כ-input אופציונלי.	לא להשתמש בו ב-MVP pre-run model אלא אם שדות מסוימים מוכחים כזמינים לפני ההרצה.	הוא Runtime data ל-pre-run leakage prediction.	הופך Early-run prediction להרחבה נפרדת.
Instance count	מופיע כ-feature אפשרי.	להשתמש רק אם הוא planned/known-before-run; אחרת להסיר.	Actual number of instances עלול להיווצר במהלך ההרצה.	Feature Availability Matrix יהפוך לשער חובה.
Heavy-job thresholds	יחס Job כבד מול האוכלוסייה.	כל quantile/threshold יחושב על Train בלבד ויישמר ב-artifact.	חישוב על כל הדאטה הוא leakage עקיף.	Preprocessing reproducible וללא test contamination.
Historical failure rate	מוצע "מבנה דומה" ללא הגדרה.	להתחיל מ-lagged history לפי stable Job key; רק אם קיים key similarity-based history הוא extension.	הגדרה לא חדה עלולה להדליף מידע או להיות לא reproducible.	MVP פשוט יותר וניתן להסבר.
XGBoost	מוגדר כמודל מרכזי מראש.	XGBoost הוא candidate challenger; נבחר רק אם הוא משפר metric + business utility על Validation.	Complexity אינה מטרה בפני עצמה.	Model selection becomes evidence-based.

נושא	החלטה נוכחית	החלטה מוצעת	סיבה	השפעה
Threshold	מוגדר לפי עלויות וקיבולת.	לבחור על Validation בלבד, להקפיא, ואז למדוד פעם אחת על Test.	אופטימיזציה על Test היא leakage של evaluation.	מגן על אמינות התוצאה.
RAG	לא הוגדר בפועל בתוכנית v2.	לא להכניס ל-MVP כרגע; לבצע RAG Gate לאחר core product.	אין בתוכנית corpus של incidents/runbooks/logs עם Ground Truth retrieval.	שומר על core DS/DE חזק ומונע GenAI קישוטי.
Kaggle Notebook	מופיע כתוצר סופי.	נקי ב- Notebook repo; רק Kaggle publishing אם הוא מוסיף ערך.	Kaggle אינו חלק הכרחי מארכיטקטורת production-minded project.	מוריד deliverable לא חיוני.

מה נשמר ללא שינוי מהותי

הבעיה העסקית, Job-level target, pre-run prediction moment, Temporal Split, PR-AUC/Recall/Precision, threshold as a business decision, SHAP, FastAPI, Streamlit, scenario-based savings, Data Leakage rules ו-MVP-first approach.

Executive Summary | 02

מה המוצר עושה, למי, ואיזו החלטה הוא משפר

הבעיה

Data Jobs שנכשלים מתגלים לרוב לאחר שהכשל כבר התרחש, ולכן צוותי Data/Operations נאלצים להגיב: לחקור, לתקן, להריץ מחדש ולנהל השפעות downstream. התהליך יקר בזמן, יוצר עומס תפעולי ועלול לעכב pipelines ו-DWH.

הפתרון

DataFlow Sentinel בונה data pipeline שמאגד Task-level data ל-Job-run level, יוצר leakage-safe features הזמינים לפני ההרצה, מחשב `risk_probability`, מסביר את גורמי הסיכון המרכזיים, ומתרגם את התחזית ל-prioritized risk queue ול-business decision בעזרת threshold ו-cost scenarios.

המשתמש וההחלטה

המשתמש העיקרי הוא Data/Operations engineer או on-call analyst. ההחלטה שהמערכת תומכת בה: אילו Jobs מתוכננים לבדוק לפני ההרצה, לפי רמת הסיכון, קיבולת הצוות ועלות צפויה של כשל לעומת עלות חקירה.

הערך הטכני והעסקי

- Data Engineering: ingestion, Raw/Staging/Gold, dbt, data tests, orchestration, idempotency ו-data contracts.
- Data Science: imbalanced classification, temporal validation, leakage-safe feature engineering, threshold optimization ו-explainability.
- ML Product: model artifact, FastAPI, dashboard, validation, testing ו-observability.
- Business/Product: alert capacity, prevention-rate assumptions, scenario-based savings ו-transparent decision logic.

One-sentence project pitch

An end-to-end platform that scores job failure risk before execution and turns predictions into operational decisions.

Interview pitch - 30-45 seconds

גרסה לראיון

בניתי מערכת End-to-End שמטרתה לזהות Data Jobs בסיכון לפני שהם מתחילים לרוץ. התחלתי מנתוני Task, בניתי Job-run data model ו-features שזמינים רק לפני ההרצה כדי למנוע Data Leakage, אימנתי Baseline, Logistic Regression ו-XGBoost עם Temporal Split, ובחרתי threshold לפי Recall, Precision, Alert Rate ועלות עסקית ולא לפי 0.5. את המודל הגשתי דרך FastAPI ו-Streamlit, עם SHAP ו- business simulator. הרעיון הוא לא רק לתת model score, אלא לתעדף עבודה תפעולית ולהראות Expected Savings תחת הנחות שקופות.

Business Problem | 03

מהו הכאב, מי מושפע, ואיפה prediction שונה מ-prevention

מרכיב	הגדרה
Business pain	כשל של Job עלול לעכב pipelines, DWH ותהליכים downstream ולהוביל לחקירה והרצה חוזרת.
Reactive workflow	alert/log -> engineer investigates -> <- Job נכשל fix/retry -> downstream recovery.
Proactive workflow	Upcoming Job -> risk score -> prioritized review -> targeted check -> run / reschedule / adjust plan.
Primary stakeholder	Data/Operations engineer, platform engineer, data pipeline owner.
Secondary stakeholder	Team lead / engineering manager שמנהל קיבולת, reliability ועלויות.
Decision supported	איזה Job לבדוק לפני הרצה וכמה התראות הצוות מסוגל לטפל בהן.
Prediction	הערכת הסתברות/סיכון שה-Job ייכשל.
Prevention	פעולה תפעולית שמונעת כשל בפועל. אין להסיק prevention ישירות מ-TP.

גבול מקצועי

True Positive הוא "כשל שזוהה מראש", לא "כשל שנמנע". בסימולטור: Prevented Failures = Detected x Assumed Prevention Rate`. ה-Prevention Rate הוא הנחה תפעולית גלויה, לא metric של המודל.

Project Objectives | 04

יעדים נפרדים לכל שכבה כדי שהפרויקט לא יהפוך ל-model-only demo

תחום	Objective
Data Engineering	לבנות ingestion reproducible; Raw immutable layer; Job-run model; dbt tests; idempotent pipeline; clear data contracts.
Data Science	להגדיר leakage-safe target/features; temporal evaluation; להתמודד עם class imbalance; להשוות baselines; להפיק calibrated/ranked risk.
ML / Serving	לשמור model artifact + preprocessing; להגיש prediction through FastAPI; validation/error handling; explainability per prediction.

תחום	Objective
Business / Product	להפוך risk score ל- risk queue; threshold based on business constraints; scenario-based savings; transparent assumptions.
Optional GenAI / RAG	רק לאחר RAG Gate: retrieve relevant incidents/runbooks and ground suggested investigation steps with citations.

Success Criteria | 05

Acceptance criteria מדידים ללא יעדים שרירותיים

קטגוריה	קריטריון קבלה
Technical	Pipeline רץ end-to-end מאותו source file עד duplicate logical rows; API/dashboard; rerun אינו יוצר failures logged clearly.
ML	המודל הנבחר עדיף על no-skill baseline ועל business heuristic ב-PR-AUC וביכולת Test final; ranking נמדד פעם אחת לאחר freezing.
Data Quality	one row per Job run; target valid; no Gold grain הוא future columns; schema/tests pass; failed tests block downstream training.
Product	Operator יכול לראות risk queue, לפתוח Job detail, להבין main factors ולשנות business assumptions ללא שינוי קוד.
Business Value	Threshold table מציג alerts, false alerts, missed failures ו-Expected Savings לפחות ב-Conservative/Base/Optimistic scenarios. אין claim על real company savings.
RAG - if implemented	Retrieval corpus אמיתי/מסומן synthetic; retrieval benchmark מוגדר; grounded answer sources cited; abstains when evidence is insufficient.

Metric rule

לא נקבע יעד כמו "PR-AUC > 0.80" לפני שנראה את base rate וה-Success הוא separability. improvement ברור מול honest final test + decision utility + baselines.

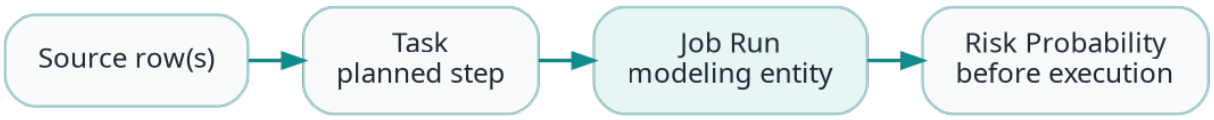
Data Understanding | 06

מה ידוע כרגע ומה מחייב validation מול הקבצים בפועל

Known from baseline document

פריט	מצב נוכחי
Primary source	`batch_task.csv` - מתואר כמכיל תכנון, DAG ותוצאת Task.
Optional source	`batch_instance.csv` - מתואר כ-runtime / extension; לא core MVP feature source.
Reported volume	14,295,731 Task rows נסרקו בעבר ב-chunks - needs revalidation.
Reported failed Tasks	83,276 Failed tasks. המסמך מציג 0.58% ובמקום אחר 0.59%; לא נשתמש בשיעור עד recomputation.
Reported resource missingness	1,345 missing resource values ב- final Tasks - needs revalidation.
Known field concepts	job_name`, Task status, `time_start`, `time_end`, planned ` CPU/Memory, DAG/dependencies, Task type. Exact schema: UNKNOWN.

Source -> Task -> Job data model



המודל העסקי אינו "Job name" אלא Job run. אם אותו `job_name` מופיע לאורך זמן, כל הרצה צריכה לקבל key נפרד. דרך יצירת ה-key היא UNKNOWN עד שנבדוק את ה-Schema וה-timestamps בפועל.

Data inventory - fields to validate

ישות	Grain	מפתחות	זמנים	Target/סטטוס	הערות
Source row	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	צריך לזהות האם כל row הוא Task definition, Task instance או event.
Task	Task within a Job run	Task key - UNKNOWN	planned/actual times - validate	Task status	Status מיועד לבניית label בלבד אם הוא final.
Job run	One scheduled/executed Job occurrence	job_run_id` - ` UNKNOWN	prediction time + run start	`failed_job`	Core modeling grain.

ישות	Grain	מפתחות	זמנים	Target/סטטוס	הערות
Instance	Runtime execution record	UNKNOWN	runtime timestamps	runtime outcome	Extension only for `.early-run`

UNKNOWN - needs validation

Exact column names, unique identifiers, time units, relationship between `job_name` and repeated runs, dependency encoding, task/instance distinction, and whether scheduled start exists before .execution

Prediction Design | 07

הגדרה פורמלית של entity, target, moment, horizon ו-leakage boundary

רכיב	החלטה
Prediction entity	.Job run
Positive class	failed_job = 1` if at least one Task in the same Job run has ` .final failed status
Negative class	failed_job = 0` if all Tasks in the Job run complete ` .successfully, subject to exact status semantics validation
Prediction moment	.Immediately before Job execution / release to run
Prediction horizon	From pre-run scoring to completion of that Job run; exact .clock horizon varies by Job
Feature availability rule	A feature is eligible only if its value is known at prediction .time
Aggregation rule	Task-level planned data -> deterministic Job-run .aggregates
Split rule	Chronological; preprocessing and historical features .respect time

Feature Availability Matrix

Feature	?Available before Job	Leakage Risk	?Use	Reason
`task_count`	Likely yes - validate	Low	YES if planned	DAG/task plan should be .known before run
Task type counts / ratios	Likely yes - validate	Low	YES if planned	Captures composition and .complexity
Dependency count / density	Likely yes - validate	Low	YES	DAG structure should exist .before execution
sum/max/mean ` `plan_cpu	Likely yes - validate	Low	YES	Planned resources are pre-run by definition, but .schema must confirm
sum/max/mean ` `plan_mem	Likely yes - validate	Low	YES	.Same as planned CPU
Missing resource flags	Yes if missingness visible pre-run	Low	YES	Missing planning metadata can be predictive and .operationally actionable
Scheduled start hour/day	UNKNOWN	Low	YES if scheduled time is pre-known	Temporal risk without .using outcome timestamps

Feature	?Available before Job	Leakage Risk	?Use	Reason
Planned concurrent Jobs	UNKNOWN	Medium	MAYBE	Only if schedule of peer jobs is available before prediction.
Historical failure rate	Yes, from prior runs only	High	YES if stable key exists	Must use lag/expanding history; never include current/future run.
Consecutive prior failures	Yes, prior runs only	High	MAYBE	Useful reliability signal if repeated Job identity is valid.
job_name` raw `	Yes	Medium	NO	Identifier can memorize entities and reduce generalization.
Final Task/Job status	No	Critical	TARGET ONLY	Direct leakage.
time_end` / actual ` duration	No	Critical	NO	Known only after execution.
Actual runtime CPU/Memory	No	Critical	NO in MVP	Use only in early-run extension.
Actual instance count	UNKNOWN	High	NO unless planned	Could be produced after runtime begins.

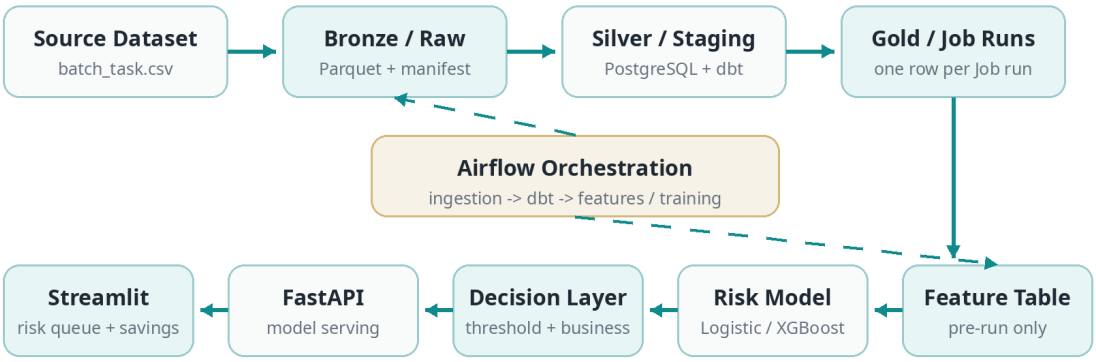
Leakage gate

כל feature חדש חייב לעבור את הטבלה הזו לפני שהוא נכנס ל-Feature Pipeline. אם זמינותו לפני הרצה אינה מוכחת - ברירת המחדל היא EXCLUDE.

Data Engineering Architecture | 08

ארכיטקטורה שמציגה יכולת DE אמיתית בלי כפילויות מיותרות

המלצה: לשמור את Airflow + Parquet + PostgreSQL + dbt + Python + Docker, אבל לתת לכל רכיב תפקיד חד. אין צורך ב-MLflow, Kafka, Spark או Vector DB ב-MVP.



Component rationale

Component	Decision	Why it exists	MVP role
Python	KEEP	Chunked ingestion, validation, feature/model code, API integration.	Core
Parquet	KEEP	Immutable raw snapshot + efficient local analytics on large source file.	Bronze
PostgreSQL	KEEP	Structured relational layer for dbt models, tests and dashboard/API metadata.	Silver/Gold store
dbt	KEEP	SQL transformations, lineage-style model structure and data tests.	Task -> Job-run model
Airflow	KEEP, narrowly	Orchestrates repeatable ingestion -> dbt -> feature/model tasks and records failures.	Workflow orchestration
Docker Compose	KEEP	Reproducible local stack for Postgres, Airflow, API, dashboard.	Packaging/demo
`batch_instance.csv`	DEFER	Runtime data conflicts with pre-run MVP; useful for early-run extension.	Extension
RAG stack	DEFER	No validated corpus yet.	Extension

Pipeline contract by component

Stage	Input	Transformation	Output / Storage	Validation	Failure / Idempotency
Ingestion	`batch_task.csv`	Chunk read + type-safe normalization + ingestion metadata	Partitioned Parquet Bronze + manifest	Schema presence, row counts, parse failures, checksum	Fail-fast on contract break; skip/reuse same checksum on rerun.
Staging	Bronze Parquet	Canonical names/types/status mapping	`stg_batch_tasks` in PostgreSQL	not_null on keys where justified; accepted statuses	Rebuildable model; no manual mutation.
Job model	Staging tasks	Group to one Job run; build `failed_job`	`fact_job_runs`	unique Job-run key, target validity, relationship checks	dbt failure blocks features/training.
Feature build	Gold Job runs + prior history	Leakage-safe aggregates / lagged features	`ml_job_features` / Parquet artifact	finite values, availability rules, temporal checks	Deterministic rebuild from source + config.
Training	Feature table	Temporal split, preprocessing, models, threshold analysis	model + preprocessor + metadata JSON	baseline comparison, reproducibility, no test tuning	Artifact written only after checks pass.
Serving	Frozen artifacts	Request validation + scoring + explanations	FastAPI response / prediction log	schema validation, version compatibility	4xx for bad input; 5xx logged; health endpoint.

Mermaid source

```
flowchart TD
    A[batch_task.csv] --> B[Bronze: Parquet + ingestion manifest]
    B --> C[Silver: PostgreSQL + dbt staging]
    C --> D[Gold: fact_job_runs - one row per Job run]
    D --> E[Leakage-safe feature table]
    E --> F[Logistic Regression / XGBoost]
    F --> G[Threshold + Business Simulator]
    G --> H[FastAPI]
    H --> I[Streamlit: Risk Queue / Job Detail / Savings]
    J[Airflow] -. orchestrates .-> B
    J -. orchestrates .-> C
    J -. orchestrates .-> E
```

Data Layers | 09

ברורים tests-ו grain עם Bronze -> Silver -> Gold

Layer	Purpose	Grain	Schema concept	Transformations	Tests
Bronze / Raw	Source-preserving raw ingest	Source row / chunk	Original fields + `ingested_at`, `ingest_run_id`, `source_checksum`, `source_file`	No business transformations; only parse normalization where unavoidable	File readable; expected columns; row count; checksum uniqueness
Silver / Staging	Canonical task data	One canonical Task/source row	Standardized types/statuses/time units; source keys preserved	Rename/cast/normalize; no target aggregation	Accepted values; basic null rules; timestamp parse validity
Gold / Job Runs	Business modeling entity	One Job run	job_run_id`, timing ` context, task counts, planned resources, `failed_job `	Task -> Job-run aggregation; target creation	Unique Job-run key; target in {0,1}; group consistency
ML Feature Table	Model-ready snapshot	One Job run	Only pre-run features + target + split timestamp	Train-safe transformations separated where needed	No prohibited columns; finite values; time-order rules

Core Gold contract

`fact_job_runs` חייב להיות one observation per Job run - לא `one row per `job_name. אם לא ניתן לזהות repeated runs בבטחה מה-Schema, עוצרים ומגדירים key לפני modeling.

Data Quality Strategy | 10

שלושה שערים: ingestion, dbt, ML validation

Ingestion validation

- File exists and is non-empty.
- Expected columns / schema version present.
- Chunk parse failures counted and logged.
- Source checksum and row count stored in manifest.
- Duplicate ingestion detected before writing a new logical snapshot.
- Schema change causes explicit failure, not silent coercion.

dbt tests

- `not_null` only on fields proven mandatory.
- `unique` on canonical Task key and Job-run key after validation.
- `accepted_values` for normalized status/target.
- `relationships` Task -> Job run where models are separated.
- Timestamp validity: start <= end only where both are actual and used for label QA, never as features.
- Custom tests for impossible planned CPU/Memory values only after domain validation.

ML data validation

- .No feature from prohibited post-run column list
- .No preprocessing fit on Validation/Test
- .Historical features use strictly earlier timestamps
- .Feature schema/version equals training contract at serving time
- .Target distribution and split sizes logged each run
- .No NaN/inf reaches the model unless the estimator intentionally supports them

EDA Plan | 11

כל גרף צריך לענות על שאלה שמובילה להחלטה

Question	Metric	Visualization	Possible Decision
מהו שיעור הכשל ברמת **Job run**?	Count + failure rate by split/time	Base-rate table, bar chart	בחירת metrics, class weights, threshold expectations.
האם **failures** מרוכזים בזמן?	Failure rate by planned hour/day	Heatmap / line chart	להוסיף temporal features או shift לבדוק.
האם **Job complexity** קשור לסיכון?	Failure rate by task_count/dependency bins	Binned plots + distributions	להנדס complexity features.
האם **planned resources** מבדילים **failures**?	CPU/MEM distributions by target	Log-scale hist/box + quantiles	Transformations, outlier policy.
האם **missing planning** קשור לכשל **metadata**?	Failure rate conditional on missing flags	Grouped bars	Keep missingness flags / operational recommendation.
האם יש **repeated Job history**?	Runs per stable key + prior failure patterns	Frequency table / lag plots	להחליט אם historical reliability features אפשריים.
האם הדאטה משתנה לאורך זמן?	Feature/target distribution by period	PSI-like summaries / temporal plots	Validation design, drift awareness.
האם **feature** חשוד ב-**leakage**?	Correlation/perfect separation + availability audit	Audit table, target-by-feature	Exclude before modeling.
האם **outliers** הם **errors** או **real heavy jobs**?	Quantiles + source checks	Log-scale plots	No blind deletion; cap/transform only if justified.

EDA rule

EDA final conclusions יחושבו על Job-run grain. Task-level EDA נשמר כרקע להבנת המקור, לא כראיה ישירה לביצועי ה-Job model.

Feature Engineering | 12

פיצ׳רים ניתנים להסבר, זמינים מראש ומחושבים בצורה reproducible

Group	Feature	Definition	Availability	Leakage	Intuition
Planned resources	`total_planned_cpu`	Sum planned CPU across Tasks/instances as defined by schema	Pre-run if planned	Low	Large planned demand may stress allocation.
Planned resources	`max_task_planned_cpu`	Max Task planned CPU	Pre-run if planned	Low	Single heavy Task may dominate risk.
Planned resources	`total_planned_mem` / max / mean	Job aggregate of planned memory	Pre-run if planned	Low	Resource intensity.
DAG / complexity	`task_count`	Number of planned Tasks in Job run	Pre-run	Low	More components can mean more failure opportunities.
DAG / complexity	`dependency_count` / density	Edges and normalized dependency density	Pre-run if DAG known	Low	Complex coordination can increase fragility.
DAG / complexity	root/dependent ratios	Counts/ratios of roots and dependent Tasks	Pre-run	Low	Structural signal beyond raw size.
Temporal	`start_hour`, `start_day`	Derived from planned/scheduled start	Only if planned start known	Low	Captures temporal operating patterns.
Load context	`planned_jobs_in_window`	Number of other scheduled Jobs around same window	Only if schedule available	Medium	Potential contention signal.
Historical reliability	`prior_failure_rate`	Failures / prior runs for stable key using strictly earlier records	Pre-run	High	Captures reliability history without current outcome.
Historical reliability	`prior_run_count`	Count of earlier runs	Pre-run	High	Supports confidence/regularization of history.
Missingness	`missing_plan_cpu_flag`, `missing_plan_mem_flag`	Known missing planning data	Pre-run	Low	Metadata quality itself may be operationally meaningful.
Transform	log1p_* for skewed counts/resources	Train-defined deterministic transform	Pre-run	Low	Stabilizes heavy-tailed distributions, especially Logistic.

Historical feature algorithm

```
sort by prediction_time
for each stable_job_key:
    prior_run_count[t]   = count(runs where time < t)
    prior_failure_count[t] = sum(failed_job where time < t)
    prior_failure_rate[t] = prior_failure_count / prior_run_count

Never include the current run or any future run.
```

Critical guardrail

Stable Job key הוא UNKNOWN כרגע. אם `job_name` אינו stable identity או אם run times אינם אמינים, historical features לא נכנסים ל-MVP.

Modeling Strategy | 13

Incremental ladder: baseline -> transparent model -> nonlinear challenger

Stage	Model	Purpose	Selection rule
Baseline 0	Majority / no-skill	Shows why Accuracy can look high under imbalance.	Reference only; must never be final model.
Baseline 1	Business heuristic	Simple rule using Train-derived complexity/resource quantiles and/or missingness.	Creates an operationally interpretable non-ML benchmark.
Model 1	Logistic Regression	Transparent coefficients; strong baseline; easy to explain.	Keep as benchmark even if XGBoost wins.
Model 2	XGBoost	Captures nonlinearities and interactions.	Select only if it improves validation ranking/utility materially.

Class imbalance

- Do not use Accuracy as the primary metric.
- No oversampling before temporal split. Default MVP: class weights / `scale_pos_weight` + threshold tuning; consider resampling only if evidence justifies it.
- PR-AUC is the primary ranking metric because the positive class is expected to be rare.

Validation strategy

- Sort by validated prediction/run time.
- Reserve the latest period as untouched Test.
- Use an earlier Validation window; if enough periods exist, prefer expanding-window temporal validation before final freeze.
- All imputers, scalers, encoders, quantile thresholds and model hyperparameters are learned from Train/Validation only.

Model selection criteria

Criterion	Role
PR-AUC	Primary ranking quality.
Recall / Precision	Operational detection quality at candidate thresholds.
Alert Rate	Team capacity / workload.
Calibration	Required if risk probabilities are shown/interpreted quantitatively.
Expected Net Savings	Business utility under transparent scenarios.

Criterion	Role
Complexity / maintainability	Tie-breaker: choose simpler model if performance is .effectively equivalent

Evaluation Framework | 14

Temporal evaluation + frozen threshold + one final Test

Chronological split

Train -> Validation -> Test. Random split is not the default because the product predicts future Job runs from past data; random mixing can overestimate generalization when behaviors and .workloads change over time

Metrics

- .PR-AUC - primary ranking metric •
- .Recall for Failed Jobs - proportion of failures detected •
- .Precision for Failed Jobs - proportion of alerts that are true failures •
- .F1 - summary only; not the sole decision rule •
- .Confusion Matrix - TP/FP/FN/TN at operational threshold •
- .Alert Rate - alerted Jobs / scored Jobs •
- .Calibration curve + Brier score if probability semantics matter •

Threshold Evaluation Table - required output

Threshold	Recall	Precision	Alerts	False Alerts	Missed Failures	Alert Rate	Expected Savings
0.10	TBD	TBD	TBD	TBD	TBD	TBD	TBD
0.20	TBD	TBD	TBD	TBD	TBD	TBD	TBD
0.30	TBD	TBD	TBD	TBD	TBD	TBD	TBD
…	…	…	…	…	…	…	…

Threshold decision protocol

1. .Generate candidate thresholds on Validation only
2. For each threshold compute Recall, Precision, Alert Rate, FP, FN and scenario-based Expected .Savings
3. .Apply operational constraints: e.g., maximum alerts/day - UNKNOWN until chosen
4. .Discuss the tradeoff with the user before freezing the threshold
5. .Freeze threshold + assumptions, then evaluate once on the untouched Test
6. .Report both model quality and business outcome; do not retune after seeing Test

Explainability | 15

לענות על "למה ה-Job הזה מסוכן?" בלי לטעון לסיבתיות

Model	Explanation method	Output
Logistic Regression	Standardized coefficient signs/magnitudes + per-row contribution if practical.	Global transparent baseline.
XGBoost	SHAP TreeExplainer.	Top positive/negative contributions per Job + global summary.
Any model	Deterministic playbook mapping from validated risk factors to checks.	Operational action, separate from statistical explanation.

Explanation policy

SHAP אומר אילו features תרמו לתחזית ביחס ל-baseline של המודל. הוא אינו מוכיח שה-feature גרם לכשל. נוסח dashboard/API צריך להשתמש ב-"main risk factors" או "model drivers", לא "root cause".

Business Value Simulator | 16

מתרגמים confusion matrix להחלטות ועלויות - עם הנחות גלויות

Core formulas

```
Detected Failures = True Positives
Prevented Failures = True Positives * Assumed Prevention Rate
Gross Avoided Cost = Prevented Failures * Cost per Failure
Investigation Cost = Alerts * Cost per Alert Investigation
Expected Net Savings = Gross Avoided Cost - Investigation Cost
Savings per 1,000 Jobs = Expected Net Savings / Scored Jobs * 1000
```

Configurable inputs

Input	Meaning	Source / status
Cost per failure	Estimated downtime + rerun + engineering/downstream impact.	UNKNOWN; user-controlled scenario assumption.
Investigation cost per alert	Engineer time and/or direct cost to inspect one alert.	UNKNOWN; user-controlled.
Assumed prevention rate	Fraction of correctly detected failures that intervention can actually prevent.	UNKNOWN; scenario assumption, not model metric.
Alert capacity	Maximum alerts team can investigate per day/window.	UNKNOWN; business constraint.
Engineering hourly cost	Optional detail for converting investigation time to cost.	UNKNOWN.
Rerun compute cost	Optional component of failure cost.	UNKNOWN.

Required scenarios

Scenario	Prevention assumption	Cost assumption	Purpose
Conservative	Low	Low/moderate failure cost + full investigation cost	Stress-test whether value survives cautious assumptions.
Base / Realistic	Mid-range	Balanced documented assumptions	Primary product story; still labeled Estimated/Expected.
Optimistic	Higher	Higher preventable impact	Shows upside, never presented as guaranteed savings.

No fabricated ROI

אם Base scenario יוצא שלילי - מציגים זאת. הפרויקט חזק יותר כשהוא מראה באילו תנאים המערכת כדאית ובאילו לא, במקום להכריח ROI חיובי.

Product Layer - FastAPI | 17

Serving contract שמפריד בין model prediction לבין UI

Endpoints

Endpoint	Purpose	MVP
`POST /predict`	Score one Job run from validated pre-run inputs.	MUST
`POST /predict/batch`	Score a list/table of upcoming Jobs for risk queue.	SHOULD
`GET /health`	Liveness + loaded model version.	MUST
`GET /model-info`	Model version, threshold, feature schema, training timestamp.	SHOULD

Conceptual response schema

```
{
  "job_run_id": "...",
  "risk_probability": 0.82,
  "risk_level": "HIGH",
  "alert_threshold": 0.31,
  "main_risk_factors": [
    {"feature": "dependency_density", "direction": "higher_risk", "contribution": 0.14}
  ],
  "recommended_checks": ["Review dependency-critical tasks before release"],
  "model_version": "dataflow-sentinel-xgb-v1",
  "explanation_note": "Model drivers are associations, not proven causes."
}
```

Validation / error handling

- Pydantic request schema; reject missing required pre-run fields with 422.
- Reject unknown model schema/version incompatibility clearly.
- No raw `job_name` used as model feature even if supplied for display/logging.
- API logs request ID, model version, latency, score and validation outcome - not raw sensitive payload by default.
- Recommended checks in MVP come from deterministic playbook, not an LLM.

Dashboard | 18

כל עמוד צריך לאפשר שאלה + החלטה + פעולה

Page	User Question	Visualization / Data	Action Enabled
Overview	מה מצב הסיכון הכולל עכשיו?	Jobs scored, alerts, risk distribution, failure base-rate context	Adjust view/window; open risk queue.
Risk Queue	איזה Jobs לבדוק קודם?	Sortable table: risk probability, level, top factor, planned start	Open Job detail / mark reviewed in demo.

Page	User Question	Visualization / Data	Action Enabled
Job Detail	למה ה-Job סומן?	Feature values + SHAP drivers + deterministic recommended checks	.Investigate targeted factors
Model Performance	האם המודל עובד ובאיזה tradeoff?	PR curve, threshold table, confusion matrix, calibration if used	Understand reliability / choose .threshold in validation mode
Business Value Simulator	מה הערך תחת הנחות שונות?	Scenario inputs + gross/net savings + alerts/failures	Compare threshold/cost .scenarios
Data Quality	האם הנתונים אמינים?	Latest ingestion, tests passed/failed, row counts, schema version	.Block/diagnose pipeline issues
RAG / Incident Assistant	מה אירועים דומים אומרים?	Only after RAG Gate	.Extension only

RAG Architecture Evaluation | 19

שתי ארכיטקטורות והחלטה מפורשת

Architecture A - without RAG

Risk Model -> SHAP -> deterministic playbook -> operator review. זה מספיק ל-MVP ומחובר ישירות לדאטה שהוגדר בתוכנית.

Architecture B - with RAG

Risk Model -> SHAP -> retrieve historical incidents/runbooks/log snippets -> grounded LLM explanation + suggested investigation steps + citations. המודל הסטטיסטי נשאר predictor; ה-LLM אינו מנבא failure.

Dimension	Without RAG	With RAG
Complexity	Moderate	High - corpus, chunking, embeddings, retrieval eval, prompting, citations.
Required data	Task/Job data + playbook rules	Meaningful incident/runbook/log corpus with metadata and source identity.
User value	Prioritized risk + reasons + fixed checks	Context-rich investigation guidance from historical evidence.
Evaluation	ML + product + business metrics	All core metrics + Recall@k/nDCG or labeled retrieval set + groundedness/citation checks.
Portfolio value	Strong DS + DE + MLOps story	Adds GenAI only if retrieval is credible and measured.
Implementation effort	Fits MVP	Can distract from core if corpus is weak.

Recommendation: DO NOT USE RAG YET

אין corpus מאומת של historical incidents, runbooks או troubleshooting materials הנוכחיים ב-documents. לכן RAG לא נכנס ל-MVP. לאחר שה-core עובד, נפתח RAG Gate. אם נשתמש ב-synthetic incident corpus, הוא יסומן במפורש כ-synthetic portfolio extension ולא יוצג כ-real company data.

RAG Gate - criteria before implementation

1. Meaningful corpus exists and each document has stable source metadata.
2. User question is explicit: retrieve similar incident/runbook to investigate a risky Job.
3. Simple SQL/filtering baseline is compared with semantic/hybrid retrieval.
4. Embedding/retrieval strategy is chosen after corpus analysis, not by default.
5. Retrieval quality benchmark is defined (e.g., labeled query -> relevant document).
6. LLM answer must cite retrieved sources and abstain when evidence is insufficient.
7. RAG demonstrably improves operator usefulness enough to justify complexity.

Repository Structure | 20

מקצועי, מודולרי, אבל לא מפוצל יתר על המידה

```
dataflow-sentinel/
├── README.md
├── pyproject.toml
├── .env.example
├── .gitignore
├── configs/
│   ├── project.yaml
│   └── business_scenarios.yaml
├── data/
│   ├── raw/              # gitignored source files / local mount
│   ├── bronze/           # parquet snapshots / local artifacts
│   └── samples/          # tiny safe sample for tests/demo if allowed
├── src/dataflow_sentinel/
│   ├── ingestion/
│   ├── features/
│   ├── modeling/
│   ├── evaluation/
│   ├── business/
│   └── common/
├── dbt/
│   └── dataflow_dbt/
├── airflow/
│   └── dags/
├── api/
│   └── app/
├── dashboard/
├── notebooks/
│   ├── 01_data_understanding.ipynb
│   └── 02_job_level_eda.ipynb
├── artifacts/            # models/metrics locally; large files gitignored
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── model/
│   └── api/
├── docs/
│   ├── master_specification.pdf
│   ├── data_dictionary.md
│   ├── decision_log.md
│   └── interview_story.md
├── docker/
├── docker-compose.yml
└── .github/workflows/ci.yml
```

הערה: `/rag` לא קיים ב-MVP. נוסיף אותו רק אם RAG Gate מאושר. כך מבנה ה-repo משקף את ה-Scope האמיתי.

Testing Strategy | 21

בדיוק מה בודקים - לא רק "יש tests"

Test Type	What exactly	Pass condition
Unit	status normalization, Job aggregation, feature calculations, business formulas, threshold helpers.	Fast, deterministic tests on tiny fixtures.
Data / dbt	unique keys, target values, relationships, accepted statuses, schema checks.	Pipeline blocks on failure.

Test Type	What exactly	Pass condition
Integration	Raw sample -> staging -> Gold -> feature .table	Expected row counts/keys on fixture .dataset
Model	No future columns, deterministic split, model beats no-skill reference on controlled fixture/smoke dataset, artifact schema .present	.Protects methodology and artifact contract
API	Valid request, missing field, bad type, .health, model version, batch response	.FastAPI TestClient
Dashboard smoke	App starts and can load sample/model .metadata	.Basic integration smoke, not pixel tests
RAG - if added	retrieval relevance, citation presence, .abstention on no evidence	.Extension only

Reproducibility | 22

notebook magic מישהו אחר צריך להיות מסוגל לשחזר את התוצאה בלי

- Use `pyproject.toml` with pinned compatible dependency ranges or lock file; document Python .version
- All paths and thresholds/configurable assumptions live in YAML/env, not hard-coded .notebooks
- .Random seeds set for model training where relevant; report non-deterministic components
- .Temporal split boundaries are saved in run metadata
- Model artifact bundle includes preprocessor, feature order/schema, threshold, model version .and training metadata
- .Docker Compose recreates PostgreSQL/Airflow/API/dashboard services for demo
- Raw large dataset is not committed; README explains how to obtain/place it and validates .checksum if allowed
- Notebooks are exploratory/presentation artifacts; production pipeline logic lives under `src/` .and dbt

Logging & Monitoring | 23

production extensions מול MVP observability

Area	MVP	Extension
Pipeline logging	ingest run id, checksum, row counts, duration, validation failures	central log backend / alerting
API logging	request id, model version, latency, status code, risk score summary	distributed tracing / SLOs
Prediction logging	prediction timestamp, job_run_id/display id, probability, threshold, model version	join with eventual outcomes for live performance
Data drift	offline comparison across historical periods in report	scheduled drift metrics / alerts
Model drift	not live unless future outcomes arrive	rolling PR-AUC/Recall/calibration monitoring
Alert monitoring	dashboard alert volume under test scenarios	production on-call integration
Retraining	manual reproducible training command	scheduled/triggered retraining with approval gate

Portfolio realism

.בחברה production deployment של claim לא ,reproducible local product הוא MVP-ה
.extension architecture-כ מוצגים ולכן future stream + outcomes דורשים "Monitoring/retraining "live

Project Roadmap | 24

אבני דרך קטנות, בדיקות קבלה, והרחבות שלא מעכבות MVP

Phase	Goal / Tasks	Deliverable	Acceptance	Dependencies	Main Risk
.0 Specification & Data Validation	Approve scope; inspect actual schema; define Job-run key, prediction timestamp, status mapping, feature .availability	Data dictionary + .contract + decision log	No UNKNOWN in core grain/target; sample .aggregation validated	.Access to source files	Ambiguous run .identity
.1 Reproducible Ingestion	Chunk load; Parquet Bronze; manifest/checksum; .logs	Idempotent ingestion .command/DAG	Same source rerun does not duplicate logical snapshot; row counts .match	.Phase 0	Large-file memory / type .parsing
.2 Silver/Gold Job Model	dbt staging; `fact_job_runs`; target; .tests	.Postgres models + dbt tests	One row per Job run; target validated; tests .green	Reliable key/status .mapping	.Aggregation mistakes
.3 Job-level EDA & Feature Table	EDA questions; leakage audit; planned/DAG/temporal/ history features where .valid	ml_job_features` + ` .EDA notebook/report	All used features pass availability matrix; distributions .documented	.Gold model	Historical feature .leakage
.4 Modeling & Threshold	Baselines; Logistic; XGBoost; temporal validation; calibration check; threshold/business .table	Frozen model artifact + metrics + threshold .decision	Model selection justified; threshold chosen on Validation; .untouched Test report	.Feature table	Overfitting / poor base .rate
.5 API + Dashboard + Business Simulator	FastAPI; Streamlit; .explanations; scenarios	Working local .product/demo	Risk queue works; response schema validated; assumptions .editable	.Frozen artifact	.UI scope creep
.6 Hardening & Portfolio	Tests, CI, Docker Compose, README, architecture, interview .story	.Portfolio-ready repo	Clean setup + demo + test pass + .documentation	.Core MVP	.Time pressure
.7 Optional Extensions	Early-run or RAG only .after gate	Separate extension .branch/module	Must not weaken core .MVP	.MVP complete	Complexity without .evidence

Roadmap rule

אם שלב מתעכב: מורידים extension, לא מורידים leakage checks, temporal evaluation, data tests או business transparency.

Portfolio / Interview Value | 25

איזו יכולת כל רכיב מוכיח

Component	Data Science Skill	Data Engineering Skill	Business Skill
Chunked ingestion + Parquet	Large-data handling, reproducible preprocessing	Ingestion design, storage format, idempotency	Reliability / operational discipline
PostgreSQL + dbt layers	Data understanding for modeling	Data modeling, SQL transformations, tests	Trusted metrics / governed logic
Job-run Gold model	Correct analytical grain	Dimensional/business grain thinking	Aligns data to decision unit
Leakage-safe features	Feature engineering, temporal reasoning	Feature pipeline reproducibility	Credible predictions
Temporal split	Generalization to future	Time-aware pipeline boundaries	Avoids misleading results
Baselines + Logistic + XGBoost	Model comparison, imbalance handling	Artifact pipeline	Evidence-based complexity
Threshold table	Precision/Recall/business tradeoffs	Configurable decision logic	Capacity and cost-aware decisions
SHAP	Explainability	Serving explanation artifacts	Operator trust / actionability
Business simulator	Utility modeling	Reusable business config	Scenario-based ROI, transparent assumptions
FastAPI	Inference contract	Service interface / validation	Productization
Streamlit	Model communication	App integration	Operator decision UX
Tests + Docker + CI	Model/data quality discipline	Engineering reproducibility	Portfolio credibility
RAG if justified	Retrieval evaluation, grounding	Document pipeline / vector retrieval	Faster incident investigation with evidence

Interview Check themes to master

- ?Why aggregate Task -> Job run
- ?Why temporal split instead of random split
- ?Why is Accuracy misleading
- ?How did you prevent Data Leakage
- ?Why is threshold a business decision
- ?Why use both Parquet and PostgreSQL
- ?Why dbt here
- ?Why is SHAP not root-cause analysis
- ?Why should an LLM not predict Job failure
- ?Under what conditions would RAG add value

Final MVP Definition | 26

מה חייב, רצוי, אופציונלי ומחוץ לתחום - Scope contract

MUST HAVE

.Validated Job-run grain, target and prediction moment •
.Chunked ingestion with Bronze Parquet + manifest/checksum/idempotency •
.PostgreSQL + dbt staging/Gold models and data tests •
.One row per Job run in Gold and model-ready leakage-safe feature table •
.Job-level EDA tied to decisions •
.Baseline 0 + business heuristic + Logistic Regression + XGBoost candidate •
.Chronological Train/Validation/Test; no Test tuning •
.PR-AUC, Recall, Precision, F1, Confusion Matrix, Alert Rate •
.Threshold analysis that includes business scenarios and team capacity assumption •
.SHAP (if XGBoost wins) or transparent equivalent for final model •
.`FastAPI `POST /predict` + `GET /health •
Streamlit risk queue + Job detail + model performance + business simulator + data-quality •
.status
.Tests, Docker Compose, README, architecture diagram, decision log, interview story •

SHOULD HAVE

.Batch prediction endpoint •
.Calibration analysis and calibration fix if needed •
.GitHub Actions CI •
.Model-info endpoint and artifact metadata •
.Historical features if stable repeated-run key is validated •
.Planned-load context if schedule data is truly available pre-run •

OPTIONAL

.`Early-run prediction using `batch_instance.csv •
.Time-to-failure / failure-within-window target •
.Scheduled drift monitoring / retraining •
.RAG incident/runbook assistant after RAG Gate •
.Synthetic incident corpus only as clearly labeled extension if no real corpus exists •

OUT OF SCOPE for MVP

.Automatic infrastructure changes or self-healing actions •
.Claiming that alerts prove failures were prevented •
.Real company savings claims without real cost/outcome data •
.Kafka/Spark/Kubernetes/MLflow only for buzzword value •
.LSTM/GRU unless sequential runtime modeling becomes a validated extension •
.LLM as failure predictor •

Initial Decision Log + Project State | 27

נקודת פתיחה שתישמר לאורך כל הפרויקט

Project Decision Log

ID	Decision	Reason	Date/Phase	Status
D-001	Prediction grain = Job run, not Task and not raw job .name	Matches business decision .and repeated executions	Phase 0	PROPOSED - approve
D-002	Target = at least one failed Task within the same Job .run	Aligns failure outcome to .Job-level decision	Phase 0	PROPOSED - validate statuses
D-003	MVP prediction occurs .before execution	Creates proactive rather .than reactive product	Phase 0	PROPOSED - approve
D-004	Only pre-run available .features are eligible	Strict Data Leakage .prevention	Phase 0	MANDATORY
D-005	Raw = Parquet; transformed = PostgreSQL/dbt; Airflow .orchestrates	Each technology has distinct portfolio/engineering .purpose	Phase 0	PROPOSED
D-006	Model ladder: no-skill -> heuristic -> Logistic -> .XGBoost	Evidence-based complexity .and interview clarity	Modeling	PROPOSED
D-007	Threshold selected on Validation using model + business constraints, then .frozen	Avoids default 0.5 and Test .tuning	Evaluation	MANDATORY
D-008	Business savings are Expected/Scenario-based .only	No fabricated company .ROI	Business	MANDATORY
D-009	RAG excluded from MVP until corpus/retrieval gate .passes	No validated corpus yet; .protect core scope	Phase 0	RECOMMENDED

PROJECT STATE

```
Current phase: Specification / approval
Completed: Baseline plan reviewed; Master Specification drafted
Current datasets: batch_task.csv referenced; batch_instance.csv optional - actual local
availability needs validation
Validated assumptions: Business goal, Job-level direction, pre-run MVP, temporal evaluation
requirement
Open assumptions: Exact schema, Job-run key, status semantics, prediction timestamp, pre-run
feature availability, cost/capacity assumptions
Key decisions: See D-001 through D-009
Files created: Master Project Specification v3 (this document)
Known issues: baseline task failure rate inconsistency; current Job-level failure rate unknown
Next milestone: Validate the actual source data and lock the Data Contract before coding the
pipeline
```

Unresolved assumptions

- יש לאמת מהו המפתח הייחודי של Job run מתוך עמודות המקור בפועל.
- יש לזהות איזה timestamp מייצג את רגע המידע הזמין לפני ההרצה.
- יש למפות אילו Task statuses נחשבים terminal failure ואילו terminal success.
- יש לאמת ש-planned CPU/Memory ו-DAG אכן ידועים לפני execution לכל Job.
- יש לקבוע האם instance count הוא planned או runtime-derived.
- יש לאמת שקיים stable Job identity לצורך historical features ללא Leakage.
- יש להגדיר operational alert capacity שה-threshold צריך לכבד.
- יש להגדיר הנחות cost ו-prevention rate עבור Conservative/Base/Optimistic scenarios.
- יש לבדוק האם קיים corpus אמיתי של incidents/runbooks/logs שמתאים ל-RAG.

Recommended MVP scope

Recommended MVP

batch_task` -> Bronze Parquet -> PostgreSQL/dbt Job-run Gold -> leakage-safe feature table -> ` Baselines/Logistic/XGBoost -> temporal evaluation -> threshold + business simulator -> FastAPI -> .Streamlit -> tests/Docker/README. `batch_instance` and RAG remain extensions

RAG recommendation

DO NOT USE RAG YET

ה-core project כבר מציג DS + DE + ML Systems + business value. RAG ייכנס רק אם נמצא corpus אמיתי/מוצדק וניתן יהיה למדוד retrieval quality; אחרת הוא יחליש את האמינות במקום לחזק אותה.

First implementation milestone after approval

Milestone 0 - Data Contract & Grain Validation

לפני כל pipeline code: נטען sample מהמקור, נציג exact schema, נזהה את Job-run key ואת prediction timestamp, נמפה statuses, ונמלא Feature Availability Matrix על בסיס העמודות האמיתיות. רק אחרי שה-checkpoint הזה עובר נבנה ingestion.

STOP FOR APPROVAL

המסמך מגדיר את ה-**Scope**. אין להתחיל **implementation** עד לאישור או תיקונים.